

Learning to Adapt: Self-Improving Web Agent via Cognitive-Aware Exploration

Weile Chen^{1†} Bingchen Miao^{1†} Qifan Yu^{1†} Wendong Bu¹ Guoming Wang^{1*} Wenqiao Zhang^{1*}
 Shengyu Zhang¹ Juncheng Li^{1*} Siliang Tang¹

¹Zhejiang University

Corresponding authors: {Nb21013, wenqiaozhang, junchengli}@zju.edu.cn

Abstract

Recent advances in Multimodal Large Language Models (MLLMs) have led to promising progress in web agents. However, existing web agents often rely on hand-crafted execution pipelines or expensive expert trajectories, limiting their adaptability to complex, dynamic environments. To address these challenges, we propose **SCALE** (Self-Cognitive-Aware Learning and Exploration), which leverages three adversarial roles—Selector, Predictor, and Judger to autonomously discover the agent’s limitations and expand its cognitive boundaries through environmental exploration. Moreover, we propose **SCALE-Hop**, a graph exploration strategy that facilitates global planning and helps agents avoid local exploration traps. To further support learning, we construct **SCALE-20k**, a large-scale dataset collected from 19 real-world websites, containing diverse task types and structured demonstrations generated from SCALE’s exploration traces. Experimental results show that our approach significantly improves the performance and generalization of multiple MLLMs in various web environments. Our framework offers a scalable and generalizable solution for building truly autonomous and adaptive web agents.

1. Introduction

Recently, with the development of Multimodal Large Language Models (MLLMs), MLLM-based web agents [14, 26, 32, 33] have demonstrated promising performance in web automation tasks, like product searching [11], shopping [34], and navigation [15]. However, most existing web agents heavily rely on the pre-trained capabilities of MLLMs, which are difficult to cover large-scale and diverse web task data [2, 23]. There remains a significant gap between their prior knowledge and the highly varied and dynamic real-world web pages. To address this gap, prior

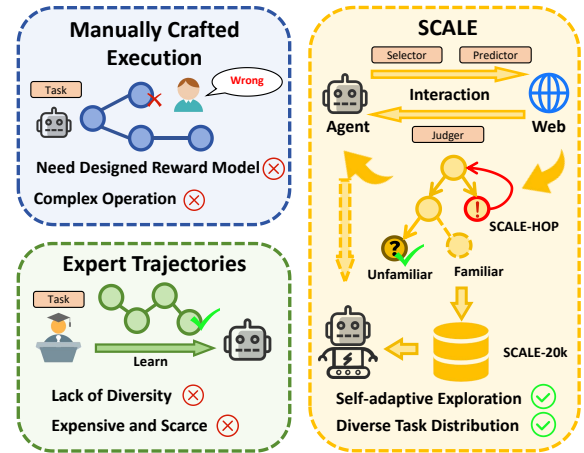


Figure 1. A comparison between prior methods and our SCALE framework. SCALE enables autonomous exploration with diverse and scalable task generation, overcoming the limitation in previous approaches.

works usually depend on the design of manually crafted execution pipelines [7, 12, 31] or on the use of human-annotated expert trajectories [3, 10, 28, 29] for fine-tuning web agents.

However, these two types of paradigms mentioned above still have notable limitations: **1) Reliance on crafted execution pipelines or expert trajectories.** These methods either rely on crafted execution pipelines for task execution or depend on costly expert trajectories to fine-tune MLLMs [16, 22, 26]. Although these pipelines and trajectories are often tailored for specific scenarios, they still struggle to cope with the unpredictability and diversity of real-world web environments [8, 23, 25]. Agents in these paradigms tend to act passively and develop dependencies on specific task flows, limiting their ability to explore unfamiliar environments and adapt to dynamic real-world websites. **2) Challenges in evaluating and expanding cognitive boundaries.** Cognitive boundaries are the limitations of an agent’s abilities, referring to items and operations that the agent struggles to understand or decide on based on prior knowledge [24]. It is significant for agents to im-

[†]Equal Contribution.

^{*}Corresponding authors.

prove their abilities. However, existing methods have neglected the exploration of their own cognitive boundaries. Therefore, how to evaluate and expand agent capabilities in a personalized and dynamic manner remains a critical challenge. **In summary**, developing general web agents requires enabling agents to proactively adapt to novel environments and continuously expanding their own cognitive boundaries, instead of relying on specific systems or predefined trajectories.

To address these challenges, we propose **SCALE** (*Self-Cognitive-Aware Learning and Exploration*), a general framework that removes reliance on expert trajectories and handcrafted pipelines. SCALE enables agents to autonomously discover and expand their cognitive boundaries through self-driven interaction in web environments. Since it operates independently of any specific model architecture, it offers broad generalizability across different models. As shown in Figure 1, the agent proactively explores without external supervision by identifying its own limitations, triggering exploratory actions in uncertain areas, and adapting based on feedback. To evaluate and refine its understanding, SCALE assigns three roles to the same MLLM: *Selector*, *Predictor*, and *Judger*. The *Selector* proposes challenging actions, the *Predictor* anticipates their outcomes, and the *Judger* verifies prediction accuracy after execution. For example, the *Selector* may choose to click a logo, believing the model is unsure about its effect. The *Predictor* then estimates the outcome based on its current knowledge, expecting nothing to happen. However, once executed, the *Judger* observes a jump to the homepage, revealing a gap in the model’s understanding. This forms a closed-loop process in which the *Selector* and *Predictor* engage in an adversarial interaction, with one aiming to challenge the model and the other aiming to predict correctly, while the *Judger* provides feedback to improve both. The iterative process allows the agent to gradually adapt to complex web environments.

Furthermore, we aim to enhance SCALE by extending its exploration capability from local interactions to global reasoning and coordination. We propose the **SCALE-Hop** strategy to complement it with global planning and control. SCALE-Hop constructs a graph representation of the agent’s interaction history, where each node represents a specific web state. This graph allows the agent to monitor its exploration coverage and make decisions at a higher level. When exploration becomes difficult or complete at a given node, SCALE-Hop evaluates the model’s performance and determines whether to exit and transition to another node that has not been fully explored. This mechanism helps the agent efficiently avoid local traps and supports a more comprehensive understanding of the environment.

Based on the raw exploration traces from the SCALE process in 19 real-world websites, we construct **SCALE-**

20k, a large-scale web task dataset. Compared to existing datasets [22, 28, 29], SCALE-20k offers not only execution trajectories but also diverse task types, including multi-step tasks, single-step decisions, and page comprehension QA, supporting more comprehensive understanding and reasoning over web content.

Through comprehensive experimentation, our SCALE framework reveals that MLLMs can proactively discover unknown aspects of the environment through self-adversarial processes and achieve self-improvement by continuously discovering and exploring these unknown aspects. Compared to strategies that rely on random exploration or follow task instructions rigidly, SCALE encourages the model to collect and update data specifically in regions it does not yet understand. This leads to more informative and error-revealing data, which better supports the model’s improvement. Combined with the global route planning of SCALE-Hop, it enables agents to move beyond shallow, localized behaviors and form a more structured and comprehensive view of complex web environments. Experimentally, SCALE achieves 231.8% improvement for InternVL2.5-8B [4] and 176.3% improvement for Qwen2.5-VL-7B [1] on average task success rate. Furthermore, the SCALE-20k dataset achieves high scores in task diversity, reasoning quality, and answer quality. Even when applied directly to other MLLMs, SCALE-20k improves their agent capabilities. These results further demonstrate the effectiveness and generality of the SCALE framework.

Our main contributions are summarized as follows:

- We propose **SCALE**, a self-driven learning and exploration framework that enables web agents to autonomously discover and expand their cognitive boundaries without relying on inference systems or expert trajectories.
- We introduce **SCALE-Hop**, a dynamically adjusted exploration strategy according to the evaluation of cognitive boundaries, which facilitates broader content discovery and promotes the collection of more diverse and robust exploration.
- We further provide **SCALE-20k**, a large-scale dataset derived from the SCALE exploration process, offering diverse task types, high-quality reasoning processes and answers to support future research on web agent understanding and reasoning.

2. Related Work

2.1. Crafted Supervision for Web Agents

Early advances in MLLM-based web agents heavily rely on two forms of external supervision: crafted execution pipelines and curated expert trajectories. Crafted pipelines—such as Tree-of-Thought [12], ReAct [31], and

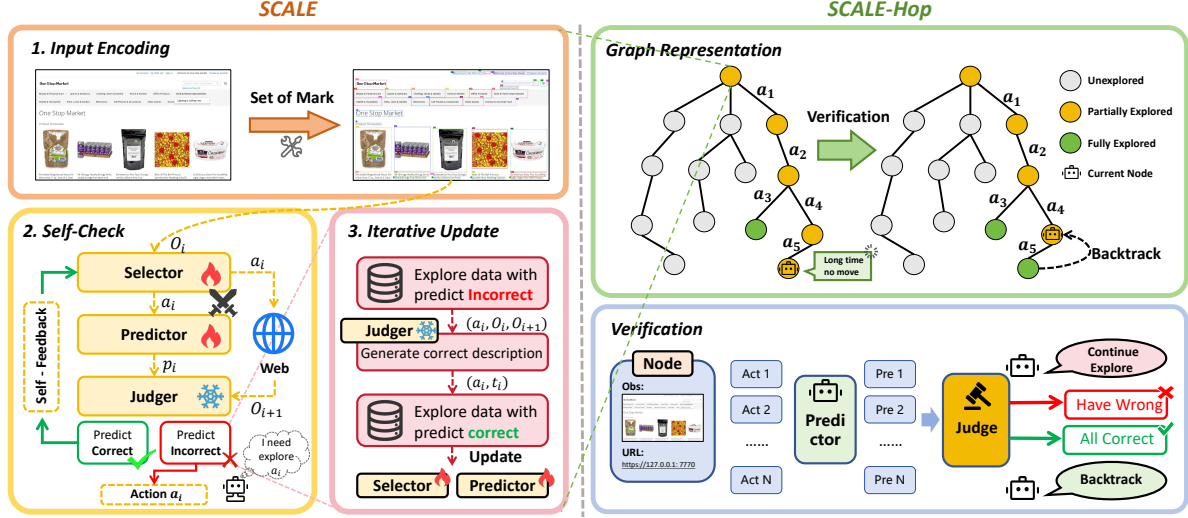


Figure 2. **The overview of SCALE and SCALE-Hop.** SCALE consists of Input Encoding, Self-Check, and Iterative Update. It enables agents to identify unfamiliar actions, verify predictions, and iteratively improve their reasoning. SCALE-Hop builds a graph to represent exploration history. It uses verification-guided backtracking to mark nodes as fully explored and guide the agent toward underexplored areas for global navigation.

world-model-based planning [7]—structure agent reasoning through explicit modules (e.g., search or simulation) to guide actions. While these enhance decision making, they require extensive manual design and lack flexibility beyond pre-defined scenarios.

Meanwhile, approaches using expert trajectories depend on large human-annotated datasets for real-world tasks. Data-centric works like Mind2Web [6], Rico [5], OS-World [28], and AGUVIS [29] advance the field, but face challenges in cost, diversity, and adaptability, resulting in passive agents rigid to new settings. In contrast, our work dispenses with handcrafted modules or costly labels, enabling agents to self-assess and extend their abilities via autonomous exploration.

2.2. Adaptive Strategy for Web Agents

To address the rigidity of manual supervision, recent methods pursue autonomous adaptation and self-improvement. Frameworks like OpenWebVoyager [9] and AgentGym [27] use iterative exploration and policy updates [17, 18], yet usually require pre-defined task templates, limiting adaptation.

Going further, exploration-based strategies such as OS-Genesis [22] and Learn-by-Interact [21] generate trajectories directly through unsupervised interactions, retroactively creating tasks and filtering via reward models. While this reduces task dependence, most methods overlook systematically probing or expanding agent cognitive boundaries. In contrast, our approach focuses on self-cognitive-aware exploration, where agents autonomously identify, diagnose, and broaden their own boundaries for robust environment adaptation.

3. Method

In this section, we first introduce the components and process of the SCALE framework, and then further introduce SCALE-Hop’s graph representation and verification-guided backtracking mechanism. Finally, we introduce the construction method and analysis of the SCALE-20k dataset.

3.1. SCALE

To move beyond handcrafted pipelines and expert trajectories, and to enable agents to proactively identify and expand their own cognitive boundaries in dynamic web environments, we propose a novel framework, **SCALE**, that enables agents to proactively explore and adapt to web environments without external supervision. In this part, we first introduce its three key components: the *Selector*, *Predictor*, and *Judge*, then describe the three core stages that facilitate cognitive boundary assessment, followed by the formalized execution process of the framework.

Selector, Predictor and Judge. The SCALE framework consists of three essential components: *Selector*, *Predictor*, and *Judge*. These roles work together to form a self-confrontation and self-verification mechanism, enabling the agent to continuously identify and focus on actions that are the most challenging to interpret within the current environment. Importantly, all three roles are assumed and executed by the agent itself, forming a fully self-contained reasoning and evaluation loop.

The *Selector* is responsible for identifying elements in the environment that are likely confusing or rarely encountered by the agent, and generating corresponding exploratory actions. For example, in a shopping page, clicking

a product is common and easy to understand. In contrast, clicking the site logo is rare and less familiar. The *Selector* prefers the latter to challenge the agent. Then, the *Predictor* is responsible for analyzing the action and anticipating its expected outcome along with an explanation. This enables the system to assess whether the agent knows about the consequences of its decisions before execution. After the action is executed, the *Judger* is responsible for comparing the predicted outcomes to the actual results to determine whether the agent truly understands the selected actions. These three roles interact in a closed-loop that forms the foundation of the SCALE.

The process of SCALE. As illustrated in Figure 2, the operation of SCALE consists of three main stages: Input Encoding, Self-Check, and Iterative Update. These stages work together to support autonomous exploration, cognitive boundary assessment, and continual adaptation.

Stage 1: Input Encoding In this stage, we adopt the commonly used Set-of-Mark (SOM) technique [30] to process Graphical User Interface (GUI) images. Unlike text-based methods that rely on accessibility trees or HTML structures, our design processes purely visual information. This choice is motivated by the fact that many real-world websites restrict access to DOM or structural metadata, whereas GUI images are always accessible. This design not only improves the generalization ability of SCALE but also aligns more closely with the way humans perceive and interact with the web through vision.

Stage 2: Self-Check In this stage, the *Selector*, *Predictor*, and *Judger* collaborate to probe the agent’s current understanding of the environment and actively search for its cognitive limits. The *Selector* deliberately proposes challenging actions, targeting uncertain or unfamiliar elements, whose outcomes the agent is least confident about. While the *Predictor* attempts to anticipate the consequence of each proposed action and explain why that outcome should occur based on its existing knowledge. This creates an internal adversarial dynamic: the *Selector* tries to expose behaviors the agent cannot yet understand, while the *Predictor* strives to refute it by accurately forecasting the true outcome. After the action is executed, the *Judger* compares the predicted and observed results to decide whether the agent truly understands the action’s effect. This self-supervised feedback loop reveals where the agent’s understanding breaks down and localizes the cognitive boundaries.

Stage 3: Iterative Update In this stage, the agent refines its reasoning through iterative self-confrontation. Based on the judgments from the *Judger*, actions with mismatched predictions are identified as failure cases, which reveal aspects of the environment that the agent does not understand. We focus on these failure cases rather than successful ones because they expose the agent’s current cognitive limitations and provide the greatest learning signal. These cases

are then used to update the *Predictor*, allowing it to learn the actual consequences of previously misunderstood actions and acquire new knowledge about the environment. Meanwhile, these cases also update the *Selector*, guiding it toward actions that remain challenging. Through this mutually reinforcing cycle, the *Selector* and *Predictor* co-evolve in a self-improving manner, continually discovering new cognitive boundaries and expanding the agent’s overall reasoning capability.

Formalized Execution Process. We denote the environment at step i as S_i , the observation as O_i , and the action as a_i , where $i \in [1, K]$, and K is the number of interaction steps in a single self-update iteration. After K steps, the agent undergoes one round of self-update. Let $j \in [1, N]$ denote the number of self-update rounds, with N as the maximum number of updates allowed. For each inner-loop iteration, the *Selector* first receives the current observation O_i and generates an action a_i :

$$a_i, r_{sel_i} = \pi_{sel_j}(O_i)$$

where r_{sel_i} is the reason for selecting a_i . This action is then passed to the *Predictor*, which predicts its outcome:

$$p_i, r_{pre_i} = \pi_{pre_j}(O_i, a_i, r_{sel_i})$$

where p_i is the prediction of a_i , r_{pre_i} is the reason for this prediction.

The action a_i is then executed in the real environment $S_{i+1} = T(S_i, a_i)$ and the agent receives the new observation $O_{i+1} = \Omega(S_{i+1})$. Next, O_i , O_{i+1} , and the predicted outcome p_i with explanation r_{pre_i} are passed to the *Judger* to determine whether the prediction was correct:

$$j_i = \text{Judger}(O_i, O_{i+1}, a_i, p_i, r_{pre_i})$$

where $j_i \in \{0, 1\}$. If $j_i = 0$, indicating that the prediction was incorrect, the action is considered unfamiliar to the agent and likely lies beyond its current knowledge boundary. In this case, we use the *Judger* to generate the true outcome and explanation:

$$t_i, r_{t_i} = \text{Judger}(O_i, O_{i+1}, a_i, r_{sel_i})$$

where t_i is the true description of the result of a_i , r_{t_i} is the true reason for this description. Then we store this exploratory data as $\text{ExploreData}_j^i = \langle O_i, a_i, r_{sel_i}, t_i, r_{t_i} \rangle$.

If $j_i = 1$, the action is deemed understood and provides no cognitive improvement. The environment is then reset to S_i , and the *Selector* generates a new action a'_i :

$$a'_i, r'_{sel_i} = \pi_{sel_j}(O_i, a_i, r_{sel_i})$$

and the above process is repeated until an unfamiliar action ($j_i = 0$) is generated.

After K steps, all collected data $\text{ExploreData}_j = \{\text{ExploreData}_j^i\}_{i=1}^K$ is used to fine-tune the *Selector* and *Predictor*:

$$\pi_{sel_{j+1}} = \text{SFT}(\pi_{sel_j}, \text{ExploreData}_j)$$

$$\pi_{pre_{j+1}} = \text{SFT}(\pi_{pre_j}, \text{ExploreData}_j).$$

Since the ability of judging is not trained during this process, the *Judge* can be considered fixed during the iterative updates. The updated policies $\pi_{sel_{j+1}}$ and $\pi_{pre_{j+1}}$ are then used for the next round of exploration, enabling an iterative self-improvement loop.

Through this process, the agent learns from unfamiliar knowledge and expands its cognitive boundaries. Unlike task-driven methods such as ReAct or tree search, SCALE performs self-driven exploration to proactively adapt to new environments and discover boundary cases, enabling higher-quality learning without extra inference-time operations. The full iterative process algorithm and more details can be found in Appendix A.

3.2. SCALE-Hop

Exploration in web environments involves not only exploring individual pages and actions but also perceiving and navigating the environment as a whole. However, relying solely on SCALE makes it difficult to obtain a global perspective for exploration and correction. Accordingly, we further propose **SCALE-Hop**, a global exploration strategy with high-level planning and dynamic control. This section introduces its Graph Representation and Verification-Guided Backtracking mechanism.

Graph Representation. As shown in Figure 2, SCALE-Hop models the agent’s exploration as a directed graph $\mathcal{G} = (\mathcal{N}, \mathcal{E})$, where each node $n_i = (O_i, u_i)$ represents a unique observation O_i and its URL u_i , and each edge $e = (n_i, a, n_j)$ denotes a transition from n_i to n_j triggered by an action a , where $i, j \in [0, K]$, and K is the total number of nodes. To distinguish different nodes, both the URL and the observation content are considered. For a candidate environment, we first check whether its URL has appeared before. If the URL is new, we directly create a new node $n_{\text{new}} = (O_{\text{new}}, u_{\text{new}})$ and add it to the graph. If the URL already exists, we compare O_{new} with all observations of nodes sharing the same URL using the Structural Similarity Index (SSIM), only when all SSIM scores are below a pre-defined threshold δ do we treat it as a new environment and insert a new node into the graph. During exploration, the graph is dynamically constructed and refined. It provides a compact and expressive way to record the agent’s exploration history, enabling efficient tracking of visited regions, recognition of novel states, and subsequent global planning over the environment.

Verification-Guided Backtracking. To further enhance the agent’s global exploration capability, we propose the

Verification-Guided Backtracking mechanism as a core component of SCALE-Hop. Each node in the exploration graph is dynamically assigned one of three states: *Unexplored*, *Partially Explored*, or *Fully Explored*, reflecting the agent’s current understanding of that local environment. The transition between these states is driven by continuous verification of prediction accuracy and exploration coverage.

When local exploration stalls or appears complete, a verification phase is triggered. The agent samples N random actions from the current node and uses the *Predictor* to predict their outcomes. If all predictions match the actual results, the node is marked as *Fully Explored*, otherwise, it remains *Partially Explored* for further investigation. Once the verification is passed, the agent then backtracks to the nearest node that is still *Unexplored* or *Partially Explored*. This mechanism balances broad exploration with strategic focus, helping the agent make steady and meaningful progress across diverse web environments.

Through the combination of Graph Representation and Verification-Guided Backtracking, SCALE-Hop performs logically consistent and globally aware exploration, avoiding getting trapped in local regions and missing globally valuable areas. This design enables the SCALE framework to maintain comprehensive and balanced coverage across the entire web environment.

3.3. SCALE-20k

Since high-quality web task data are both scarce and expensive to obtain, based on the exploration conducted by SCALE across 19 real-world websites, we construct the **SCALE-20k** dataset. We first introduce three stages for constructing the dataset, and then analyze its composition and statistics.

Dataset Construction Pipeline. As shown in Figure 3, based on the exploration data obtained from SCALE, we leverage GPT-4o to assist in task construction and validation, and construct the dataset through the following three stages.

Stage 1: Single-Step Task Generation From the SCALE exploration process, we collect numerous valid exploration actions. Then we reverse-construct a corresponding single-step task and reasoning processes based on this action.

Stage 2: Multi-Step Task Generation To reflect the complexity of real-world tasks, we extract coherent multi-step trajectories from the SCALE-Hop graph. The logically validated trajectory is then used to reverse-construct the corresponding task and reasoning process.

Stage 3: Page Comprehension QA Pair Generation To enhance the agent’s understanding of web environments, we generate question-answer pairs for each node in the SCALE-Hop graph, providing rich page-level supervision

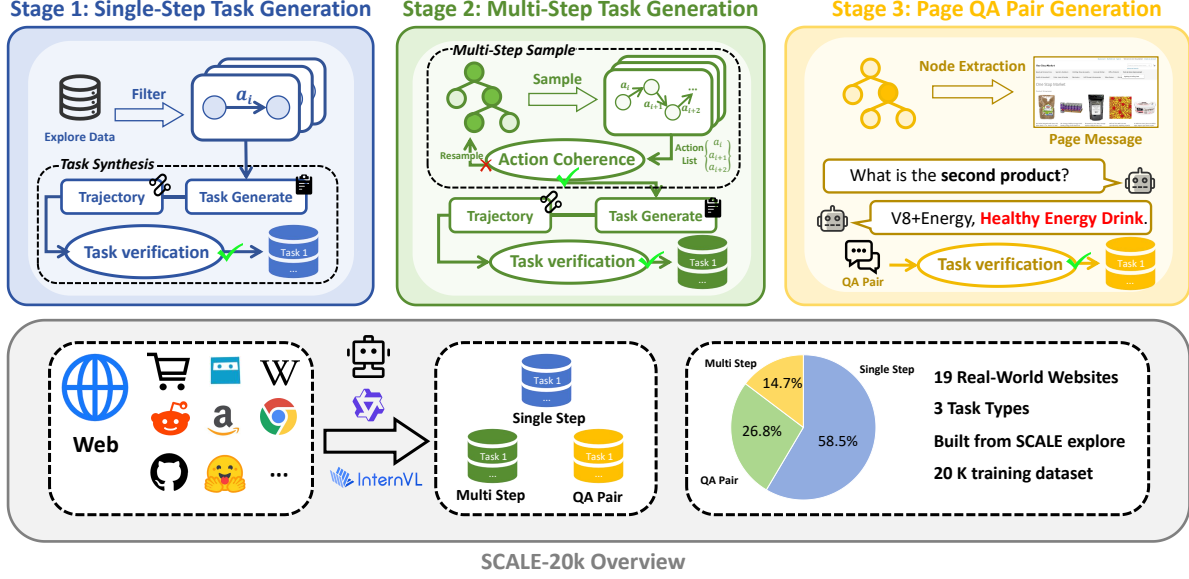


Figure 3. **Overview of the SCALE-20k construction pipeline and composition.** The dataset is constructed in three stages: 1) Single-step tasks are reverse-generated from valid exploration steps; 2) Multi-step tasks are synthesized from coherent trajectories extracted via SCALE-Hop graphs; 3) Page QA pairs are created to test content comprehension. The dataset includes 19 real-world websites and supports three task types, providing rich and diverse supervision for training web agents.

that strengthens the agent’s comprehension and reasoning ability.

Dataset Composition and Statistics. As shown in Figure 3, SCALE-20k comprises exploration data collected by Qwen2.5-VL-7B and InternVL2.5-8B across 19 real-world websites. It includes 15042 single-step tasks, 3780 multi-step tasks, and 6886 page comprehension QA tasks.

4. Experiments

In this section, we first describe the experimental setup, then present the main results, followed by an in-depth analysis of the experiments.

4.1. Experimental Setup

Baselines. We select Qwen2.5-VL-7B [1] and InternVL2.5-8B [4] as backbone models to evaluate our method. We used GPT-4o as a representative closed-source baseline. We also compare against strong open-source agents, including Augvis, a vision-only GUI agent [29], and ViGoRL, a vision-based reinforcement learning agent [20]. Beyond zero-shot settings, we consider three enhanced strategies: 1) Imitation learning on GPT-4o successful trajectories, 2) OS-Genesis exploration method [22], which constructs tasks from randomly explored trajectories, and 3) Tree Search strategy that explores multiple reasoning paths during inference time via a tree structured search. To further validate the effectiveness of our proposed SCALE-20k dataset, we additionally include a model that is entirely unrelated to our exploration

model, LLaVA-NeXT-8B [13], and fine-tune it directly on SCALE-20k.

Evaluation Benchmarks. We conduct evaluations on two widely-used web agent benchmarks: **VisualWebArena** and **WebVoyager**. VisualWebArena covers diverse domains such as shopping, classifieds, and forums, while WebVoyager includes real-world websites with complex layouts and dynamic, real-time content changes. For WebVoyager, we test on the first 11 tasks of each website to ensure consistency. To enable a fair comparison across all methods, we unify the input format by using only webpage screenshots with visual markup as the observation. All other auxiliary information, such as accessibility trees and page descriptions, is removed. Further experimental details are provided in Appendix B.

4.2. Main Results

Through comprehensive experimentation, we validate the effectiveness and generalization of our proposed method:

Effectiveness of SCALE. The SCALE framework is effective across MLLMs of different scales. As shown in Table 1, both InternVL2.5-8B and Qwen2.5-VL-7B demonstrate the ability to autonomously explore web environments in SCALE. As a result, they achieve average performance improvements of 231.8% and 176.3% over their vanilla counterparts. In addition, SCALE significantly outperforms other open-source vision-language agents such as Augvis, and ViGoRL across most environments. This validates the effectiveness of SCALE in enhancing the agent’s capabilities. Moreover, the consistent improvements across

Model	Strategy	Shopping		Classifieds		Reddit		WebVoyager	
		SR↑	AS↓	SR↑	AS↓	SR↑	AS↓	SR↑	AS↓
GPT-4o	Zero-Shot	17.2	15.5	13.7	12.5	6.7	17.6	9.6	14.7
Augvis	Zero-Shot	4.3	-	0.4	-	0.5	-	0.0	-
ViGoRL	Zero-Shot	5.6	13.4	6.9	12.4	2.9	14.6	0.6	14.9
InternVL2.5-8B	Zero-Shot	3.9	23.5	0.4	22.7	1.4	22.7	0.0	-
	GPT Trajectory Imitation	8.8	16.5	3.4	16.7	3.0	17.2	-	-
	OS-Genesis	11.6	<u>6.6</u>	3.0	<u>6.3</u>	4.3	<u>9.7</u>	0.0	-
	Tree Search	3.0	-	<u>4.3</u>	-	1.0	-	-	-
	SCALE(ours)	<u>11.0</u>	4.9	6.4	5.5	<u>3.3</u>	7.2	1.8	14.9
Qwen2.5-VL-7B	Zero-Shot	4.1	19.4	6.0	15.8	2.4	16.1	0.6	15.0
	GPT Trajectory Imitation	18.3	<u>8.5</u>	<u>10.7</u>	<u>9.2</u>	<u>3.3</u>	<u>12.0</u>	-	-
	OS-Genesis	11.2	12.7	8.6	11.3	1.4	12.7	<u>6.7</u>	15.0
	Tree Search	4.1	-	1.7	-	1.0	-	-	-
	SCALE(ours)	<u>14.4</u>	5.7	12.0	4.1	4.8	8.4	7.9	15.0
LLaVA-NeXT-8B	Zero-Shot	0	-	0	-	1.4	4.6	0.0	-
	SCALE-20k	2.1	10.1	0.8	8.2	1.9	9.6	0.0	-

Table 1. **Overall Performance Comparison Across Models and Exploration Strategies.** Success rate (SR, %) and average steps (AS) of different strategies across models on multiple web benchmarks. SR is reported as a percentage without the % symbol. The best value in each group is shown in **bold**, and the second-best value is underlined. SCALE denotes each model’s adapted performance via our SCALE procedure, while SCALE-20k refers to results obtained by training directly on the constructed SCALE-20k dataset.

Model	Strategy	Shopping	Classifieds	Reddit	Overall
Qwen2.5-VL-7B	Zero-Shot	4.5	6.0	1.9	4.1
	SCALE (20-25)	6.7	<u>9.8</u>	5.3	7.2
	SCALE (40-25)	<u>9.0</u>	9.0	4.3	<u>7.9</u>
	SCALE (60-25)	14.4	12.0	<u>4.8</u>	11.9

Table 2. **Performance of Qwen2.5-VL-7B under different exploration levels.** SR is shown as a percentage without the % symbol. SCALE (X-25) denotes X outer-loop hops and 25 inner-loop steps. Best results are in **bold**, second-best are underlined.

Strategy	Shopping		Classifieds		Reddit		Overall	
	SR	Node	SR	Node	SR	Node	SR	Node
Random Walk	14.8	207	<u>7.3</u>	<u>163</u>	3.8	<u>29</u>	<u>10.4</u>	<u>399</u>
w/o SCALE-Hop	13.5	<u>226</u>	6.0	25	7.1	26	10.1	277
SCALE	<u>14.4</u>	347	12.0	292	<u>4.8</u>	237	11.6	876

Table 3. **Ablation study of exploration strategies across three web domains.** We report task success rate (SR) and number of visited nodes (Node) to reflect effectiveness and coverage. SR values omit the % symbol. Best results are in **bold**, second-best are underlined.

different models suggest that our method is general and can be applied to a wide range of MLLM backbones.

Beyond vanilla zero-shot baselines, SCALE outperforms two mainstream improvement paradigms—expert-trajectory supervision and manually engineered reasoning pipelines. Compared with models trained on GPT-4o trajectories or OS-Genesis exploration data, SCALE achieves consistently better or competitive performance, often reach-

ing or surpassing state-of-the-art results. Its advantage is particularly notable on WebVoyager, where dynamic content and diverse page layouts make static or manually curated trajectories insufficient, highlighting SCALE’s stronger generalization and adaptability.

In terms of efficiency, we report the average number of steps (AS) taken across all tasks, regardless of success. As shown in Table 1, SCALE achieves the lowest or second-lowest AS in most domains, indicating more concise and effective reasoning paths compared to trajectory-based or tree-search baselines.

Moreover, as shown in Table 2, increasing the number of SCALE iterations leads to consistent performance gains, indicating that deeper exploration enables the agent to discover more informative behaviors and improve task success rates.

4.3. In-Depth Analysis

In this section, we further analyze how SCALE enables deeper and broader exploration.

Ablation Study: The Importance of Self-Adversarial Interaction in Exploration. We observe that the self-adversarial mechanism in SCALE plays a vital role in enhancing exploration behavior. As shown in Table 3, our method significantly outperforms random walk in both task success rate (SR) and exploration coverage. It consistently visits more unique nodes with higher diversity in URL distribution and visual layout. Many of these nodes lie in rarely

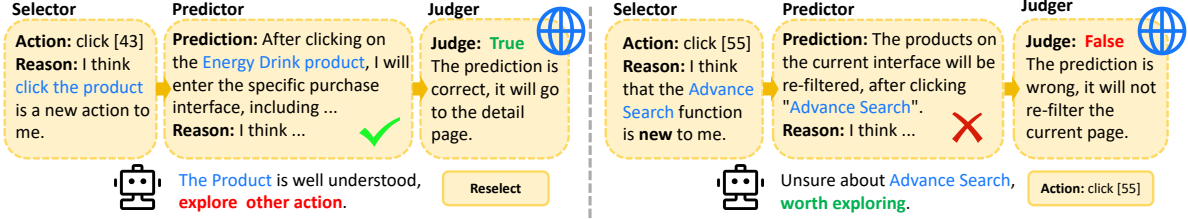


Figure 4. A case of cognitive boundary discovery by the *Selector*, *Predictor* and *Judger* in SCALE

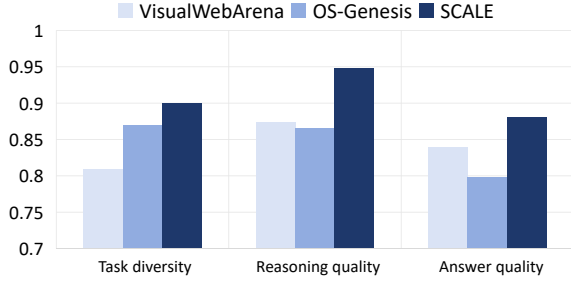


Figure 5. Comparison of SCALE-20k, OS-Genesis, and VisualWebArena across three dimensions.

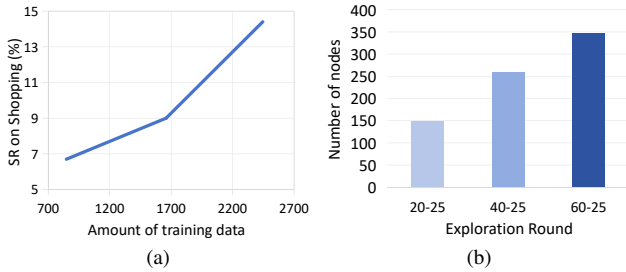


Figure 6. (a) Impact of training data size on Success Rate in shopping environment. (b) Node diversity under different exploration rounds. (X-25 indicates X outer-loops and 25 inner steps.)

accessed regions—such as footer menus, hidden tabs, and domain edges—that are often overlooked by goal-driven or randomly sampling agents. This demonstrates that self-adversarial interaction is key to enabling not only deeper exploration, but also continual self-driven improvement.

Ablation Study: Contribution of SCALE-Hop. Table 3 further highlights the impact of SCALE-Hop within the framework. Its integration yields the largest performance improvement across environments, boosting both SR and the number of visited nodes. On average, SCALE-Hop increases node coverage by 216% compared to the w/o SCALE-Hop setting, enabling the agent to escape local patterns and uncover a wider range of states. This confirms SCALE-Hop as a crucial driver of long-horizon, diverse, efficient, and self-improving exploration.

Diversity of the SCALE-20k. To evaluate dataset diversity, we compute the average cosine distance between Sentence-BERT [19] embeddings of task descriptions. In addition, we assess dataset quality using GPT-4o along two complementary dimensions, *reasoning quality* and *answer quality*, with evaluation prompts detailed in Appendix C.

As shown in Figure 5, SCALE-20k achieves strong performance across all three metrics, exhibiting higher semantic diversity and better reasoning-action correctness than other datasets. Moreover, as shown in Table 1, even a model unrelated to our framework (LLaVA-NeXT-8B) gains clear improvements when fine-tuned solely on SCALE-20k, further demonstrating the effectiveness and generality of our dataset.

Case Study. As shown in Figure 4, this case demonstrates how SCALE’s components collaboratively identify the agent’s cognitive boundary. On the left, although the selected action (clicking the product) is valid, the Predictor already understands its outcome well, suggesting this interaction is within the agent’s current knowledge and offers limited exploratory value. On the right, a new action (clicking “Advance Search”) is selected. The Predictor’s expectation is invalidated by the Judger, signaling a gap in understanding. This mismatch reveals a gap in understanding and drives targeted exploration, showcasing SCALE’s ability to recognize and expand cognitive boundaries.

Towards Lifelong Exploration via Self-Adaptation. As exploration progresses, the agent gradually improves its ability to handle complex interfaces without external supervision—each update relies solely on internal judgment and feedback. As shown in Figure 6, both task success rate and exploration diversity increase steadily. This illustrates a promising direction toward lifelong exploration learning, where agents continuously adapt to evolving web environments through self-driven feedback loops.

5. Conclusion

In this work, we propose **SCALE**, a self-driven framework that enables web agents to autonomously explore and expand their cognitive boundaries without relying on hand-crafted pipelines or expert demonstrations. It uses the *Selector*, *Predictor*, and *Judger* to collaboratively form internal feedback loops, enabling the agent to identify and overcome its cognitive limitations. At the global level, we design the **SCALE-Hop** strategy to guide broader and more strategic exploration across diverse web environments. Additionally, we construct **SCALE-20k**, a large-scale and diverse dataset, from SCALE trajectories. Experimental results demonstrate the effectiveness and adaptability of our approach for building general web agents.

Acknowledgment

This work was supported by the National Key Research and Development Program of China (2025ZD0123100), the Zhejiang NSF (LQK26F020001), Key R&D Program of Zhejiang (2026SDXT005), Fundamental Research Funds for the Central Universities (226-2025-00057), Ningbo Yongjiang Talent Introduction Programme (2024A-401-G), Zhejiang University Education Foundation Qizhen Scholar Foundation.

References

- [1] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 2, 6
- [2] Wendong Bu, Yang Wu, Qifan Yu, Minghe Gao, Bingchen Miao, Zhenkui Zhang, Kaihang Pan, Yunfei Li, Mengze Li, Wei Ji, et al. What limits virtual agent application? omnibench: A scalable multi-dimensional benchmark for essential virtual agent capabilities. In *International Conference on Machine Learning*, pages 5725–5748. PMLR, 2025. 1
- [3] Xuetian Chen, Hangcheng Li, Jiaqing Liang, Sihang Jiang, and Deqing Yang. Edge: Enhanced grounded gui understanding with enriched multi-granularity synthetic data. *arXiv preprint arXiv:2410.19461*, 2024. 1
- [4] Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, et al. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2024. 2, 6
- [5] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschman, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. Rico: A mobile app dataset for building data-driven design applications. In *Proceedings of the 30th annual ACM symposium on user interface software and technology*, pages 845–854, 2017. 3
- [6] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web, 2023. 3
- [7] Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, et al. Is your llm secretly a world model of the internet? model-based planning for web agents. *arXiv preprint arXiv:2411.06559*, 2024. 1, 3
- [8] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024. 1
- [9] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Hongming Zhang, Tianqing Fang, Zhenzhong Lan, and Dong Yu. Openwebvoyager: Building multimodal web agents via iterative real-world exploration, feedback and optimization, 2024. 3
- [10] Wenyi Hong, Wei Han Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14281–14290, 2024. 1
- [11] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. *arXiv preprint arXiv:2401.13649*, 2024. 1
- [12] Jing Yu Koh, Stephen McAleer, Daniel Fried, and Ruslan Salakhutdinov. Tree search for language model agents. *arXiv preprint arXiv:2407.01476*, 2024. 1, 2
- [13] Feng Li, Renrui Zhang, Hao Zhang, Yuanhan Zhang, Bo Li, Wei Li, Zejun Ma, and Chunyuan Li. Llava-next-interleave: Tackling multi-image, video, and 3d in large multimodal models. *arXiv preprint arXiv:2407.07895*, 2024. 6
- [14] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19498–19508, 2025. 1
- [15] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. *arXiv preprint arXiv:2406.08451*, 2024. 1
- [16] Bingchen Miao, Yang Wu, Minghe Gao, Qifan Yu, Wendong Bu, Wenqiao Zhang, Yunfei Li, Siliang Tang, Tat-Seng Chua, and Juncheng Li. Boosting virtual agent learning and reasoning: A step-wise, multi-dimensional, and generalist reward model with benchmark. In *Forty-second International Conference on Machine Learning*, 2025. 1
- [17] Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. Agent q: Advanced reasoning and learning for autonomous ai agents. *arXiv preprint arXiv:2408.07199*, 2024. 3
- [18] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023. 3
- [19] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics. 8
- [20] Gabriel Sarch, Snigdha Saha, Naitik Khandelwal, Ayush Jain, Michael J Tarr, Aviral Kumar, and Katerina Fragkiadaki. Grounded reinforcement learning for visual reasoning. *arXiv preprint arXiv:2505.23678*, 2025. 6
- [21] Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Sercan Ö Arik. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments. *arXiv preprint arXiv:2501.10893*, 2025. 3
- [22] Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Li

- heng Chen, Zhoumianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. *arXiv preprint arXiv:2412.19723*, 2024. [1](#), [2](#), [3](#), [6](#)
- [23] Gaurav Verma, Rachneet Kaur, Nishan Srishankar, Zhen Zeng, Tucker Balch, and Manuela Veloso. Adapta-agent: Adapting multimodal web agents with few-shot learning from human demonstrations. *arXiv preprint arXiv:2411.13451*, 2024. [1](#)
- [24] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6): 186345, 2024. [1](#)
- [25] Shuai Wang, Weiwen Liu, Jingxuan Chen, Yuqi Zhou, Weinan Gan, Xingshan Zeng, Yuhan Che, Shuai Yu, Xinlong Hao, Kun Shao, et al. Gui agents with foundation models: A comprehensive survey. *arXiv preprint arXiv:2411.04890*, 2024. [1](#)
- [26] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024. [1](#)
- [27] Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Dingwen Yang, Chenyang Liao, Xin Guo, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. Agentgym: Evolving large language model-based agents across diverse environments. *arXiv preprint arXiv:2406.04151*, 2024. [3](#)
- [28] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024. [1](#), [2](#), [3](#)
- [29] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024. [1](#), [2](#), [3](#), [6](#)
- [30] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v. *arXiv preprint arXiv:2310.11441*, 2023. [4](#)
- [31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. [1](#), [2](#)
- [32] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v (ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614*, 2024. [1](#)
- [33] Longtao Zheng, Zhiyuan Huang, Zhenghai Xue, Xinrun Wang, Bo An, and Shuicheng Yan. Agentstudio: A toolkit for building general virtual agents. *arXiv preprint arXiv:2403.17918*, 2024. [1](#)
- [34] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan
- Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023. [1](#)